



**AMERICAN
UNIVERSITY_{OF} BEIRUT**

**MAROUN SEMAAN FACULTY OF
ENGINEERING & ARCHITECTURE**

EECE 698/MECH 650: Autonomous Mobile Robotics

Assignment 4



By:

Daniel Haikal – 202403619

Suhaib Abu Raidah – 202591285

Boulos Boulos – 202310396

To:

Prof. Daniel Asmar

May 4 2026

Chapter 8: Problem 4

Assume that the received sensor observation is z . The likelihoods used by MCL are therefore

$$\ell_1 = p(z | x_1) = 0.8, \quad \ell_2 = p(z | x_2) = 0.4, \quad \ell_3 = p(z | x_3) = 0.1, \quad \ell_4 = p(z | x_4) = 0.1.$$

Equivalently, since normalized weights are unchanged by multiplying all likelihoods by the same constant, we may use the proportional likelihood vector

$$(\ell_1, \ell_2, \ell_3, \ell_4) = (8, 4, 1, 1).$$

Let N_i denote the number of initial particles sampled at location x_i . Since the initial particles are drawn uniformly from the four possible locations,

$$(N_1, N_2, N_3, N_4) \sim \text{Multinomial} \left(N; \frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4} \right).$$

For a fixed particle count vector $n = (n_1, n_2, n_3, n_4)$, the probability that the single resampled particle is at location x_i is

$$\mathbb{P}(\tilde{X} = x_i | n) = \frac{n_i \ell_i}{\sum_{j=1}^4 n_j \ell_j}.$$

Therefore the marginal probability distribution over the single new sample is

$$p_N(x_i) = \mathbb{E} \left[\frac{N_i \ell_i}{\sum_{j=1}^4 N_j \ell_j} \right].$$

Writing this expectation explicitly gives

$$p_N(x_i) = \sum_{\substack{n_1+n_2+n_3+n_4=N \\ n_j \geq 0}} \frac{N!}{n_1! n_2! n_3! n_4!} \left(\frac{1}{4} \right)^N \frac{n_i \ell_i}{\sum_{j=1}^4 n_j \ell_j}.$$

This is the exact finite- N distribution produced by the MCL resampling procedure.

(a) The true posterior is obtained from Bayes' rule. Since the prior is uniform,

$$p^*(x_i) = p(x_i | z) = \frac{p(z | x_i) p(x_i)}{\sum_{j=1}^4 p(z | x_j) p(x_j)} = \frac{\ell_i}{\sum_{j=1}^4 \ell_j}.$$

Since

$$\sum_{j=1}^4 \ell_j = 0.8 + 0.4 + 0.1 + 0.1 = 1.4,$$

we get

$$p^* = \left(\frac{4}{7}, \frac{2}{7}, \frac{1}{14}, \frac{1}{14} \right).$$

This is also the limiting distribution as $N \rightarrow \infty$, because by the law of large numbers,

$$\frac{N_i}{N} \rightarrow \frac{1}{4},$$

and therefore

$$p_\infty(x_i) = \frac{\frac{1}{4} \ell_i}{\sum_{j=1}^4 \frac{1}{4} \ell_j} = \frac{\ell_i}{\sum_{j=1}^4 \ell_j} = p^*(x_i).$$

For finite N , the exact distributions are:

N	$p_N(x_1)$	$p_N(x_2)$	$p_N(x_3)$	$p_N(x_4)$
1	0.250000	0.250000	0.250000	0.250000
2	0.368056	0.304167	0.163889	0.163889
3	0.430182	0.314463	0.127677	0.127677
4	0.466106	0.314147	0.109874	0.109874
5	0.488602	0.311471	0.099964	0.099964
6	0.503652	0.308591	0.093879	0.093879
7	0.514279	0.306032	0.089845	0.089845
8	0.522118	0.303872	0.087005	0.087005
9	0.528112	0.302070	0.084909	0.084909
10	0.532829	0.300562	0.083304	0.083304
∞	0.571429	0.285714	0.071429	0.071429

The finite- N distributions approach the true posterior, but they are not equal to it. This is the finite-sample bias of normalized importance sampling in MCL. The key issue is that

$$\mathbb{E} \left[\frac{N_i \ell_i}{\sum_{j=1}^4 N_j \ell_j} \right] \neq \frac{\mathbb{E}[N_i] \ell_i}{\sum_{j=1}^4 \mathbb{E}[N_j] \ell_j}$$

for finite N .

(b) Using the convention:

$$D_{\text{KL}}(p_N \| p^*) = \sum_{i=1}^4 p_N(x_i) \log \left(\frac{p_N(x_i)}{p^*(x_i)} \right),$$

where $\log$ denotes the natural logarithm, the KL divergences are:

N	$D_{\text{KL}}(p_N \ p^*)$
1	0.386329
2	0.129343
3	0.056319
4	0.029475
5	0.017571
6	0.011496
7	0.008050
8	0.005930
9	0.004542
10	0.003587
∞	0

The divergence decreases as N increases, and it becomes zero in the limit $N \rightarrow \infty$, because $p_N \rightarrow p^*$. If the reverse convention is used instead, namely

$$D_{\text{KL}}(p^* \| p_N) = \sum_{i=1}^4 p^*(x_i) \log \left(\frac{p^*(x_i)}{p_N(x_i)} \right),$$

then the values are:

N	$D_{\text{KL}}(p^* \ p_N)$
1	0.331573
2	0.114852
3	0.051881
4	0.027791
5	0.016804
6	0.011095
7	0.007817
8	0.005784
9	0.004444
10	0.003518
∞	0

(c) The bias comes from the fact that, for finite N , MCL uses a random empirical approximation of the prior and then normalizes random weights. A useful way to see what is required is to consider $N = 1$.

For $N = 1$, exactly one particle is sampled from the prior. After weighting and resampling, the algorithm must return that same particle, because there is only one particle available. Hence the distribution produced by the estimator is just the prior distribution. If the prior is denoted by q , then

$$p_1(x_i) = q_i.$$

However, the true posterior after observing z is

$$\pi_i = \frac{q_i \ell_i}{\sum_{j=1}^4 q_j \ell_j}.$$

Therefore, for the estimator to be unbiased even when $N = 1$, we need

$$q_i = \frac{q_i \ell_i}{\sum_{j=1}^4 q_j \ell_j}$$

for every i with $q_i > 0$. This requires the likelihood ℓ_i to be constant over every state that has positive prior probability. In other words, the observation must not distinguish between the states that can actually occur.

One sufficient modification is to change the sensor model so that the observation z is equally likely at every location. For example, suppose

$$p(z | x_1) = p(z | x_2) = p(z | x_3) = p(z | x_4) = c$$

for some constant $0 < c < 1$. Then the observation carries no information about the robot location. The posterior remains equal to the prior:

$$p(x_i | z) = p(x_i).$$

Also, because all particle weights are equal, the resampling probability for a fixed count vector becomes

$$\frac{n_i c}{\sum_{j=1}^4 n_j c} = \frac{n_i}{N}.$$

Taking expectation gives

$$\mathbb{E} \left[\frac{N_i}{N} \right] = \frac{1}{4}.$$

Thus the finite- N estimator is unbiased for every N .

A second sufficient modification is to change the state space or prior so that the robot can only be in states that already have the same likelihood. In the original sensor model,

$$p(z | x_3) = p(z | x_4) = 0.1.$$

So, for example, suppose the problem is reformulated so that the only possible robot locations are

$$X' = \{x_3, x_4\},$$

with a uniform prior over these two locations. Then the likelihood is constant on the entire state space:

$$p(z | x_3) = p(z | x_4).$$

The posterior is therefore still uniform on $\{x_3, x_4\}$, and the normalized particle weights reduce to ordinary empirical frequencies. Hence the estimator is unbiased for every finite N .

Both modifications change the problem formulation, not the MCL algorithm. In both cases, the observation is uninformative over the states that have positive prior probability, so the normalization of random particle weights no longer introduces finite-sample bias.

Chapter 9: Problem 3

A big problem in 2D mapping is the obvious property that most sensors cannot sweep the entire z-dimension from a single measurement. It is entirely possible that an obstacle, that would be right in front of the robot in a 2D map, is not detected by the sensor.

We know that the sensor opening is cone-shaped with an angle of 15° , therefore its half-angle opening is

$$\beta = 7.5^\circ$$

Furthermore, the sensor is mounted horizontally and parallel to the ground. At a radial distance r , the lower edge of the measurement cone has dropped by

$$\text{drop} = r \tan \beta$$

Let the sensor height be h_s the obstacle height be h_o , we define the height difference to be

$$\Delta h = h_s - h_o$$

For an object that is $\Delta h = 15\text{cm}$ below the sensor, we know β , and we can find the minimum distance needed between the robot and the object to be perceived:

$$r = \frac{\Delta h}{\tan \beta} = \frac{15}{\tan(7.5^\circ)} = 113.94 \text{ cm} \approx 1.14 \text{ m}$$

(a) Assuming the sensor is noise-free and the environment is static, the sensor will detect the obstacle if the object crosses the field of view of the sonar cone. If an obstacle is too low or too high for a certain in-plane radial distance to the robot, it will not be detected. This distance is

$$r_{hit} \geq \frac{h_s - h_o}{\tan \beta}$$

(b) In a standard occupancy grid map, each grid cell is modeled as either occupied or free. The binary Bayes filter updates the probability of occupancy of each cell using sensor measurements. However, this situation shows a weakness of the standard model. A cell may contain a real obstacle, but the sonar may fail to detect it because the obstacle lies below the sonar beam. Therefore, a “no detection” measurement does not necessarily mean that the space is free.

The problem is that the normal occupancy grid algorithm treats the sensor measurement as evidence about a 2D cell, while the real sensing problem is 3D. The measurement depends not only on whether the cell is occupied, but also on the height of the obstacle, the height of the sensor, the beam angle, and the distance to the obstacle. This can make the occupancy grid algorithm fail. If the robot repeatedly observes no return from the sonar, the binary Bayes filter may incorrectly reduce the occupancy probability of the obstacle cell. As a result, the map may label the cell as free even though there is actually an obstacle there.

Therefore, the Markov assumption and the simple inverse sensor model are too weak in this case. The measurement is not fully explained by the 2D occupancy state of the cell alone. Important hidden variables, such as obstacle height and sensor geometry, are being ignored.

(c) The problem comes from the fact that we treat cells that are not being measured as empty. If a robot is stationary and is continuously reading measurements and updating the belief, probabilities in the blind spots will converge to 0, while the sensor is not sweeping the entire height.

One solution would be to use a 3D representation of the map, and storing the probability of each voxel independently. The proposed algorithm keeps OGM algorithm the same, but modifies the inverse range sensor model as follows:

Algorithm inverse_range_sensor_model($m_{i,j,k}, x_t, z_t$) :

1. $r = \sqrt{(x_i - x)^2 + (y_i - y)^2 + (z_k - z)^2}$
2. $\phi = \text{atan2}(y_j - y, x_i - x) - \theta_{\text{yaw}}$
3. $\psi = \text{atan2}(z_k - z, \sqrt{(x_i - x)^2 + (y_j - y)^2}) - \theta_{\text{pitch}}$
4. $\hat{k} = \text{argmin}_j \sqrt{(\phi - \theta_{j,\text{azimuth}})^2 + (\psi - \theta_{j,\text{elevation}})^2}$
5. **if** $r > \min(z_{\text{max}}, \hat{z}_t + \alpha/2)$ **or** $|\phi - \theta_{\hat{k},\text{azimuth}}| > \beta_{\text{horiz}}/2$ **or** $|\psi - \theta_{\hat{k},\text{elevation}}| > \beta_{\text{vert}}/2$
 return l_0
6. **if** $\hat{z}_t < z_{\text{max}}$ **and** $|r - \hat{z}_t| < \alpha/2$
 return l_{occ}
7. **if** $r \leq \hat{z}_t$
 return l_{free}
8. **endif**

Key modifications include calculating the 3D norm instead of the 2D, and adding the elevation angle for the vertical opening, as well as the appropriate out-of-bounds check. This improves the mapping algorithm because the robot no longer assumes that “no sonar return” means the entire vertical column is free. Instead, it only updates the part of space that the sensor could actually observe.

Chapter 10: Problem 7

The goal is to develop a Kalman-filter SLAM algorithm in which the basic map elements are not point landmarks, but local occupancy grid maps. Since robot poses and relative transformations are nonlinear, the practical algorithm is an Extended Kalman Filter (EKF).

1. Main idea

In classical landmark-based EKF-SLAM, the state vector contains the robot pose and the positions of all landmarks:

$$\mu_t = \begin{bmatrix} x_t \\ m_1 \\ m_2 \\ \vdots \\ m_N \end{bmatrix}.$$

Here, instead of landmarks, we use local occupancy grid maps. Each local grid is treated as a rigid object whose pose is estimated by the Kalman filter. The actual grid cells are not placed inside the Kalman state, because that would make the state extremely large.

Therefore, the EKF estimates:

robot pose + poses of local occupancy grids.

Each local occupancy grid stores its own cell probabilities separately using the standard occupancy-grid mapping method.

2. State representation

Assume a planar mobile robot. The robot pose is

$$x_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}.$$

Let the pose of local grid map G_i in the global frame be

$$s_i = \begin{bmatrix} x_i \\ y_i \\ \theta_i \end{bmatrix}.$$

If there are N local grid maps, the EKF state vector is

$$\mu_t = \begin{bmatrix} x_t \\ s_1 \\ s_2 \\ \vdots \\ s_N \end{bmatrix}.$$

The covariance matrix is

$$\Sigma_t = \text{cov} [x_t, s_1, s_2, \dots, s_N].$$

Each local grid map is represented as

$$G_i = \{p(c_1), p(c_2), \dots, p(c_M)\},$$

where $p(c_j)$ is the occupancy probability of cell c_j inside the local coordinate frame of grid G_i . These cell probabilities are not part of the EKF state vector.

3. Motion prediction

The robot moves according to the nonlinear motion model

$$x_t = f(x_{t-1}, u_t) + \epsilon_t,$$

where u_t is the odometry or control input and ϵ_t is zero-mean Gaussian motion noise:

$$\epsilon_t \sim \mathcal{N}(0, R_t).$$

The local grids are static, so

$$s_i(t) = s_i(t-1).$$

Therefore, the predicted mean is

$$\bar{\mu}_t = g(\mu_{t-1}, u_t),$$

where g updates only the robot pose and leaves all local grid poses unchanged.

The covariance prediction is

$$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R'_t,$$

where G_t is the Jacobian of the motion model with respect to the full state, and R'_t is the motion-noise covariance embedded into the full state dimension.

Only the robot pose receives direct motion noise. The local grid poses remain unchanged during prediction.

4. Local occupancy-grid update

Suppose the robot is currently using local grid G_k . The robot pose relative to this local grid is

$$x_t^{(k)} = s_k^{-1} \oplus x_t,$$

where $\oplus$ denotes pose composition.

The current sensor scan z_t is inserted into G_k using an inverse sensor model. In log-odds form, the update for a cell c is

$$L_t(c) = L_{t-1}(c) + \log \frac{p(c \mid z_t, x_t^{(k)})}{1 - p(c \mid z_t, x_t^{(k)})} - L_0,$$

where

$$L_t(c) = \log \frac{p_t(c)}{1 - p_t(c)}$$

and L_0 is the prior log-odds value.

Thus, the occupancy grid is updated locally, while the EKF estimates the pose of the local grid in the global map.

5. Measurement model using scan matching

A sensor scan can be matched against the active local grid G_k . This scan matching produces an estimate of the robot pose relative to the local grid:

$$\hat{z}_t = \begin{bmatrix} \hat{x}^{(k)} \\ \hat{y}^{(k)} \\ \hat{\theta}^{(k)} \end{bmatrix}.$$

The predicted measurement from the EKF state is

$$h_k(\mu_t) = s_k^{-1} \oplus x_t.$$

The innovation is

$$r_t = \hat{z}_t \ominus h_k(\bar{\mu}_t),$$

where $\ominus$ denotes pose difference.

The innovation covariance is

$$S_t = H_t \bar{\Sigma}_t H_t^T + Q_t,$$

where H_t is the Jacobian of h_k and Q_t is the covariance of the scan-matching estimate.

The Kalman gain is

$$K_t = \bar{\Sigma}_t H_t^T S_t^{-1}.$$

The mean and covariance are updated as

$$\mu_t = \bar{\mu}_t + K_t r_t,$$

$$\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t.$$

This update constrains the robot pose relative to the active local grid.

6. Creating a new local grid

A new local grid G_{N+1} is created when one of the following conditions is satisfied:

$$\|x_t - s_k\| > d_{\max},$$

or

$$|\theta_t - \theta_k| > \theta_{\max},$$

or when the active grid becomes too large or the pose uncertainty becomes too large.

The pose of the new grid is initialized from the current robot pose:

$$s_{N+1} = x_t.$$

The EKF state is augmented as

$$\mu_t^+ = \begin{bmatrix} \mu_t \\ s_{N+1} \end{bmatrix}.$$

Since the new grid pose is initialized from the robot pose, its uncertainty must be correlated with the robot uncertainty. If

$$s_{N+1} = a(\mu_t),$$

then the augmented covariance is

$$\Sigma_t^+ = \begin{bmatrix} \Sigma_t & \Sigma_t A^T \\ A \Sigma_t & A \Sigma_t A^T + R_{\text{init}} \end{bmatrix},$$

where A is the Jacobian of $a(\mu_t)$, and R_{init} represents small initialization uncertainty.

7. Relating local grids to each other

The main difficulty is how to relate one local grid to another. This can be done by matching overlapping local occupancy grids.

Assume that local grids G_i and G_j overlap. A grid-matching algorithm can estimate the relative transformation between them:

$$\hat{z}_{ij} \approx s_i^{-1} \oplus s_j.$$

The predicted relative pose between the two grids is

$$h_{ij}(\mu_t) = s_i^{-1} \oplus s_j.$$

The innovation is

$$r_{ij} = \hat{z}_{ij} \ominus h_{ij}(\mu_t).$$

The EKF update is then

$$S_{ij} = H_{ij} \Sigma_t H_{ij}^T + Q_{ij},$$

$$K_{ij} = \Sigma_t H_{ij}^T S_{ij}^{-1},$$

$$\mu_t = \mu_t + K_{ij} r_{ij},$$

$$\Sigma_t = (I - K_{ij} H_{ij}) \Sigma_t.$$

This update is analogous to observing a landmark in classical EKF-SLAM, but here the observation is a relative pose constraint between two local maps.

This also allows loop closure. If the robot returns to a previously visited place, the current local grid can be matched with an older grid. The resulting relative-pose measurement reduces the accumulated drift.

8. Dealing with the growing number of local grids

If every local grid is kept forever inside the EKF state, the state dimension becomes

$$3 + 3N.$$

Therefore, the covariance matrix has size

$$(3 + 3N) \times (3 + 3N).$$

The computational cost grows quickly as N increases. Several strategies can be used to control this growth.

A. Keep grid cells outside the EKF

The EKF should estimate only the pose of each local grid, not the occupancy value of every cell. This reduces the number of variables from thousands or millions of grid cells to only three variables per local grid.

B. Use bounded-size local grids

Each local grid should cover only a limited area, for example

$$20\text{ m} \times 20\text{ m}.$$

When the robot leaves the area, a new local grid is created. This keeps each occupancy grid computationally manageable.

C. Match only likely overlapping grids

It is not efficient to compare every grid with every other grid. Candidate grids can be selected using spatial gating. For example, match G_i and G_j only if

$$\|s_i - s_j\| < d_{\text{gate}}.$$

A more statistically correct test is the Mahalanobis distance:

$$r_{ij}^T S_{ij}^{-1} r_{ij} < \gamma.$$

Only grid pairs satisfying this condition are considered for grid matching.

D. Merge highly overlapping grids

If two local grids have large overlap and small relative uncertainty, they may be merged. For example, merge G_i and G_j if

$$\text{overlap}(G_i, G_j) > \alpha$$

and

$$\text{uncertainty}(s_i^{-1} \oplus s_j) < \beta.$$

This reduces the number of local grid components.

E. Marginalize old local grids

Old local grids far away from the robot can be removed from the active EKF state. Their information can be preserved as fixed map information or as relative-pose constraints. The active state then contains only

$$\text{robot pose} + \text{nearby local grid poses}.$$

This produces an approximate but practical bounded-size filter.

9. Complete algorithm

The complete algorithm can be written as follows.

|